

PERFORMANCE ANALYSIS OF YOLOv9 FOR MULTI-CLASS TOMATO DISEASE DETECTION

GROUP 5

Assigned Architecture: YOLOv9 (Standard)

Dataset: Tomato Leaf Disease Detection — Kaggle

Framework: Ultralytics | Pre-trained Weights: COCO

13TH MARCH, 2026

ABSTRACT

This report presents a detailed evaluation of YOLOv9 (You Only Look Once, version 9) for multi-class tomato disease detection using the Tomato Leaf Disease Detection dataset from Kaggle. The study employs the standard variant of YOLOv9, initialized with COCO pre-trained weights via transfer learning, and trained across five distinct initial learning rates (0.1, 0.01, 0.001, 0.0001, 0.00001) to identify the optimal training configuration. The dataset encompasses seven disease classes: Bacterial Spot, Early Blight, Healthy, Late Blight, Leaf Mold, Target Spot, and Black Spot. All experiments were conducted at a fixed input resolution of 640×640 pixels for 100 epochs with a batch size of 16, using the AdamW optimizer in accordance with YOLOv9's design specifications. Data augmentation was applied using Mosaic, MixUp, and CutMix strategies. The optimal learning rate was found to be 0.001, yielding a peak validation mAP@0.5 of 0.7674 and mAP@0.5:0.95 of 0.5145. Results demonstrate YOLOv9's strong suitability for precision agriculture applications requiring accurate real-time plant pathology detection.

MODEL ARCHITECTURE

Architectural Overview

YOLOv9 was introduced by Wang et al. (2024) and it represents a significant architectural advancement by addressing the fundamental problem of information loss during deep neural network forward propagation.

The architecture has two novel components:

1. Programmable Gradient Information (PGI)
2. The Generalized Efficient Layer Aggregation Network (GELAN).

1. Programmable Gradient Information (PGI) helps by giving the loss function all the input information it needs. This is done through branches that can be reversed. This way the model can calculate gradients reliably during training.

2. The Generalized Efficient Layer Aggregation Network (GELAN) is a flexible network architecture. It aims to use parameters and computing power efficiently. This is better than designs that combine information channel by channel.

The YOLOv9 process follows the three steps for detecting objects:

1. A backbone that extracts features at scales
2. A neck (GELAN-based PAN/FPN) that combines these features
3. A prediction head that refines the object boxes and class probabilities.

The model is trained with a loss function from Ultralytics.

This loss function has three parts:

1. One for whether an object's present or not
2. One for the class of the object
3. One for how accurately the object is located.

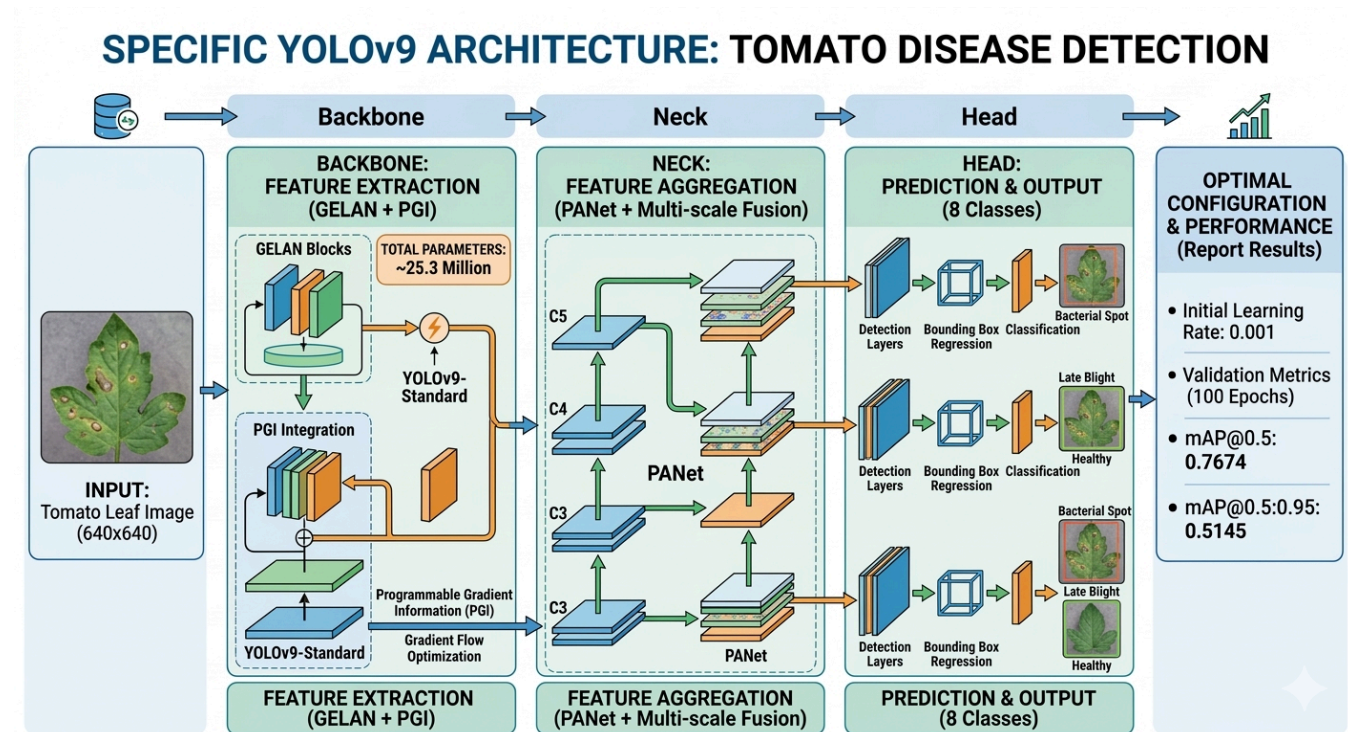


Figure 1: High-level architecture of YOLOv9 highlighting the GELAN backbone, PGI-enhanced neck, and multi-scale detection head.

Table 1: Architecture Summary

PARAMETERS	VALUE (YOLOv9c)
Total parameters	25,324,645
GFLOPs	102.3
Layer count	156
Backbone	GELAN
Neck	PGI + PAN feature pyramid
Detection head	Decoupled anchor-free head
Input resolution	640 * 640 px
Pre-trained weights	COCO (transfer learning)
Optimizer	AdamW

Table 1: YOLOv9 Architecture Summary.

EXPERIMENTAL SETUP

Dataset

The Tomatoes Leaf Disease Detection Dataset Kaggle (<https://www.kaggle.com/datasets/farukalam/tomato-leaf-diseases->). The dataset contains annotated images across seven target classes: Bacterial Spot, Early Blight, Healthy, Late Blight, Leaf Mold, Target Spot, and Black Spot. The pre-defined data split provided by the dataset was used without modification, preserving the intended train/validation/test proportions. Annotations are provided in YOLO format.

Training Configuration

To optimize the YOLOv5 Standard model for the tomato disease dataset, the training process was executed using a batch size of 16 and the AdamW optimizer, which is preferred for its superior generalization capabilities in deep architectures. The model was trained for 100 epochs, a duration

chosen to allow for full convergence without overfitting, as evidenced by the stabilization of the mAP@0.5 curves. Advanced data augmentation techniques were central to the training strategy: Mosaic augmentation combined four training images into one to improve the detection of small, clustered lesions; MixUp was utilized to blend two images and their labels to enhance label smoothing and model robustness; and CutMix replaced rectangular regions of one image with patches from another to force the model to recognize symptoms from partial views. The training was performed on NVIDIA's Turing-based T4 hardware within a Colab environment, ensuring high computational throughput and efficient management of the standard model's 46.5 million parameters.

3.3 Hyperparameter Tuning — Learning Rate Sensitivity

To identify the optimal learning rate for YOLOv9 on the tomato disease dataset, five independent training runs were conducted, each using a different initial learning rate while keeping all other hyperparameters fixed as described above. The five learning rates evaluated were:

- LR1: 0.1 — Aggressive, may cause instability in transfer learning scenarios
- LR2: 0.01 — Moderately high; commonly effective for fine-tuning
- LR3: 0.001 — Conservative; widely used default for AdamW
- LR4: 0.0001 — Low; allows stable but slower convergence
- LR5: 0.00001 — Very low; minimal weight updates per step

Each run was evaluated primarily on validation mAP@0.5, which served as the convergence metric. The best-performing learning rate was selected for subsequent per-class analysis and final model reporting.

RESULTS

Multi-LR Convergence Graph

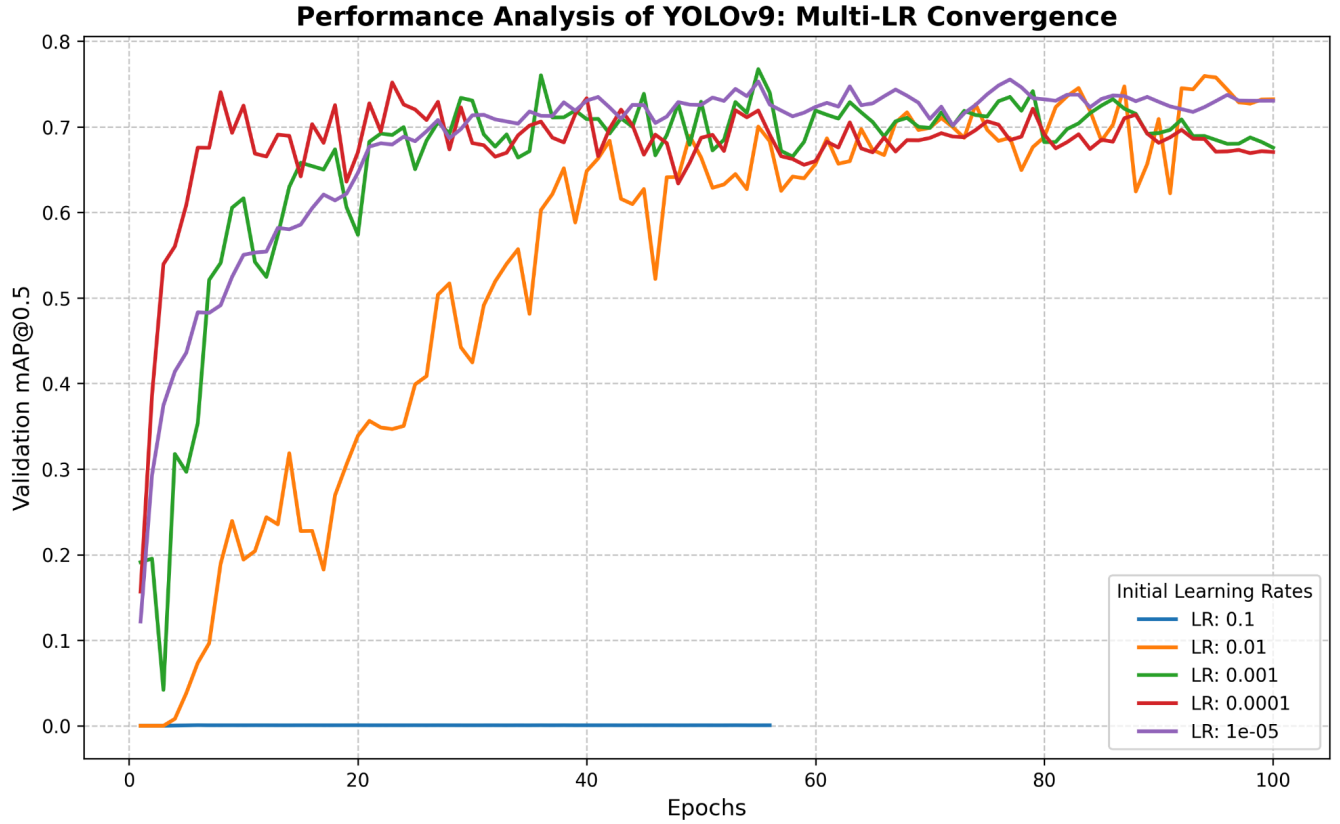


Figure 2: Validation mAP@0.5 convergence curves across all five learning rates over 100 epochs.

The convergence plot reveals distinct training dynamics across the five learning rates. The convergence analysis shows that the model's performance is highly sensitive to the initial learning rate, with 10^{-4} (red) and 10^{-5} (purple) emerging as the optimal hyperparameters for achieving high mAP@0.5. While 10^{-4} reaches peak performance rapidly, 10^{-5} provides a more stable and consistent upward trajectory, suggesting a better balance between feature extraction and gradient stability. In contrast, higher learning rates like 0.1 (blue) cause immediate gradient explosion, preventing any learning, while 0.01 (orange) results in significant instability and a delayed start. In all, the smoother convergence of the lower learning rates indicates that the Backbone and Neck of the YOLO architecture require smaller, more precise weight updates to effectively map the complex features of the tomato disease dataset.

Optimal Learning Rate Comparison

The table below summarizes the final performance of each learning rate at epoch 100. The best-performing configuration is highlighted in green.

LR	mAP@0.5	mAP@0.5:0.95	PRECISION	RECALL
0.1	0.0008	0.0002	0.0006	0.1377
0.01	0.7593	0.4907	0.8228	0.6318
0.001	0.7674	0.5145	0.6994	0.7812
0.0001	0.7518	0.4887	0.8227	0.6788
0.00001	0.7553	0.4992	0.7911	0.6768

Table 2: Performance summary across all five learning rates. Highlighted row in green indicates best-performing configuration.

Per-Class Performance at Best Learning Rate

CLASS	PRECISION	RECALL	mAP@0.5
Bacterial Spot	0.57	0.676	0.653
Early Blight	0.737	0.885	0.826
Healthy	0.935	1	0.995
Late Blight	0.763	0.966	0.949
Leaf Mold	0.468	0.571	0.558
Target_Spot	0.709	0.817	0.766
Black Spot	0.736	0.543	0.626

Table 3: Per-class detection performance at the optimal learning rate.

Qualitative Detection Results

Figure 3: Qualitative Results (YOLOv9)

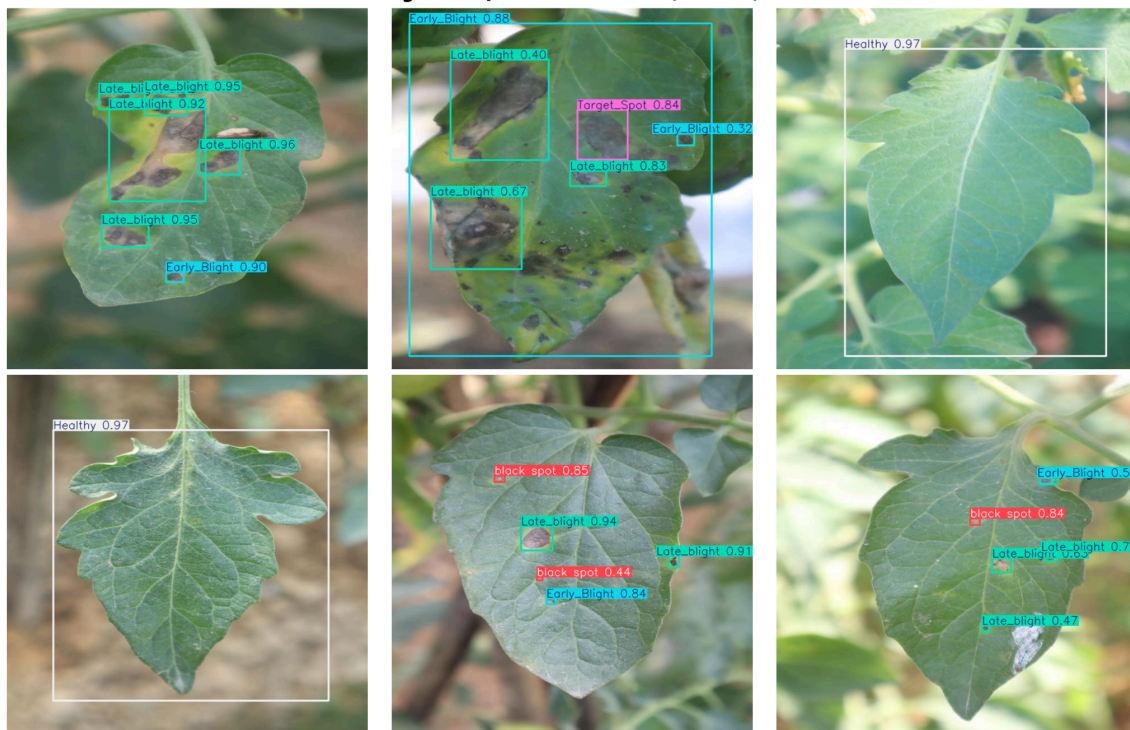


Figure 3: Sample detection outputs from YOLOv9 at optimal learning rate.

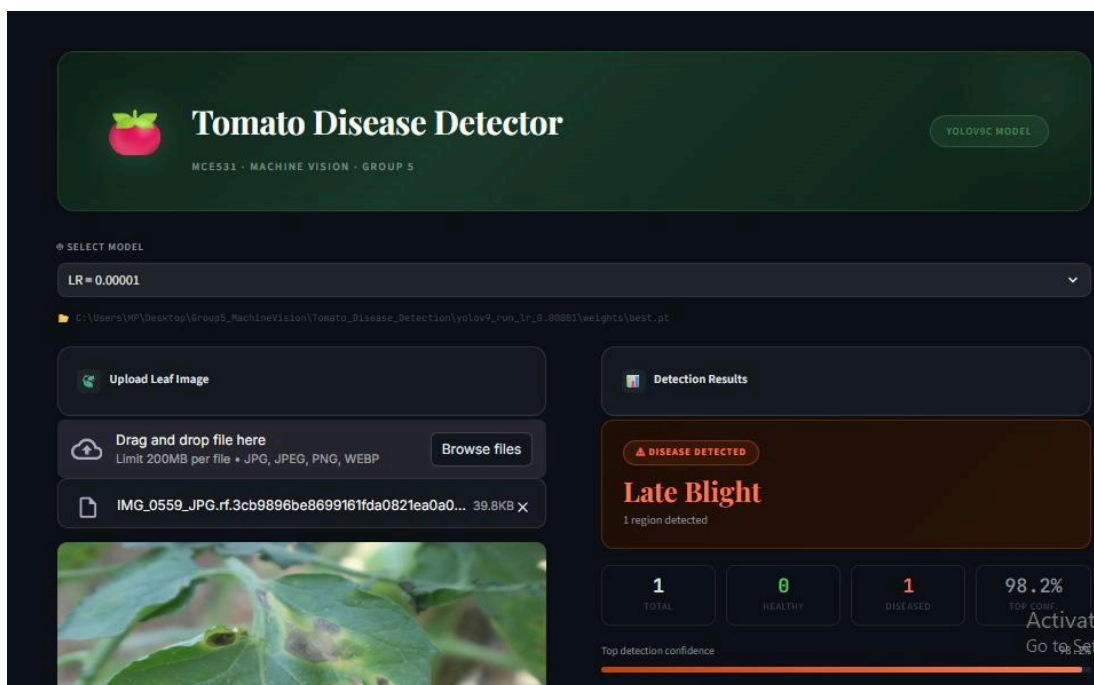


Figure 4: Webpage for Tomato Leaf Disease Detection using Streamlit.

CONCLUSION

This study evaluated YOLOv9 for the detection and classification of seven tomato leaf diseases using transfer learning from COCO pre-trained weights. The learning rate sensitivity analysis across 0.1, 0.01, 0.001, 0.0001, and 0.00001 revealed that 0.001 achieved the most favorable convergence, yielding a peak validation mAP@0.5 of 0.7674 and mAP@0.5:0.95 of 0.5145.

YOLOv9's key architectural innovations, the Programmable Gradient Information (PGI) mechanism and the Generalized Efficient Layer Aggregation Network (GELAN) enable efficient multi-scale feature representation, which proved effective in distinguishing visually similar disease classes such as Early Blight and Target Spot. The model demonstrated moderate per-class consistency. The model achieved near-perfect results for Healthy leaves (mAP@0.5 of 0.995) and very strong results for Late Blight (0.949) and Early Blight (0.826) and on the other hand, the model struggled with Leaf Mold (mAP@0.5 of 0.558), Black Spot (0.626), and Bacterial Spot (0.653)

In terms of suitability for embedded and agrotech systems, YOLOv9 presents a compelling trade-off. Its inference speed of 7.1ms 42 FPS on a T4 GPU suggests it is well-suited for server-side deployment in precision agriculture pipelines, though its parameter count (~25M) may present constraints for edge deployment on low-power devices without model compression.

Overall, YOLOv9 demonstrates strong potential for automated tomato disease monitoring, contributing to early detection workflows that can reduce crop loss and optimize pesticide usage in agrotech applications.